LABORATORIO #05

INTERFAZ GRAFICA

PRESENTADO POR:

MIGUEL ANGEL MURILLO HURTADO

JUANITA RUBIANO FERNANDEZ

PRESENTADO A:

MARIA IRMA DIAS ROZO

ESCUELA COLOMBIANA DE INGENIERIA JULIO GARAVITO

Bogota D.C

2026 – 1

CONTEXTO

Reto: Implementar el juego **EasySokoban**

El propuesto por ustedes SokobanGUI	El acordado en laboratorio Sokoban
Vista-Controlador	**Modelo**

El trabajo debe hacerse desde CONSOLA

Para la capa de presentación NO deben hacer diagramas de diseño ni pruebas de unidad

PARTE I. MAQUETA

A. Directorios

El objetivo de este punto es preparar los directorios para el juego EasySokoban

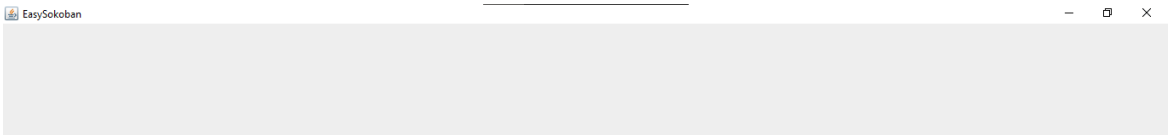
1. **Preparen un directorio llamado EasySokoban con los directorios src, bin y lib; y los subdirectorios para presentación, dominio y pruebas de unidad. Capturen una pantalla con la estructura.**

B. Ciclo 0: Ventana vacía - Salir

El objetivo es implementar la ventana principal de EasySokoban con un final adecuado desde el icono de cerrar. Utilizar el esquema de prepareElements-prepareActions.

1. **Construyan el primer esquema de la ventana de EasySokoban únicamente con el título "EasySokoban ". Para esto creen la clase SokobanGUIcomo un JFrame con su creador (que sólo coloca el título) y el método main que crea un objeto SokobanGUIy lo hace visible. Ejecútenlo. Capturen la pantalla. (Si la ventana principal no es la inicial en su diseño, después deberán mover el main al componente visual correspondiente)**

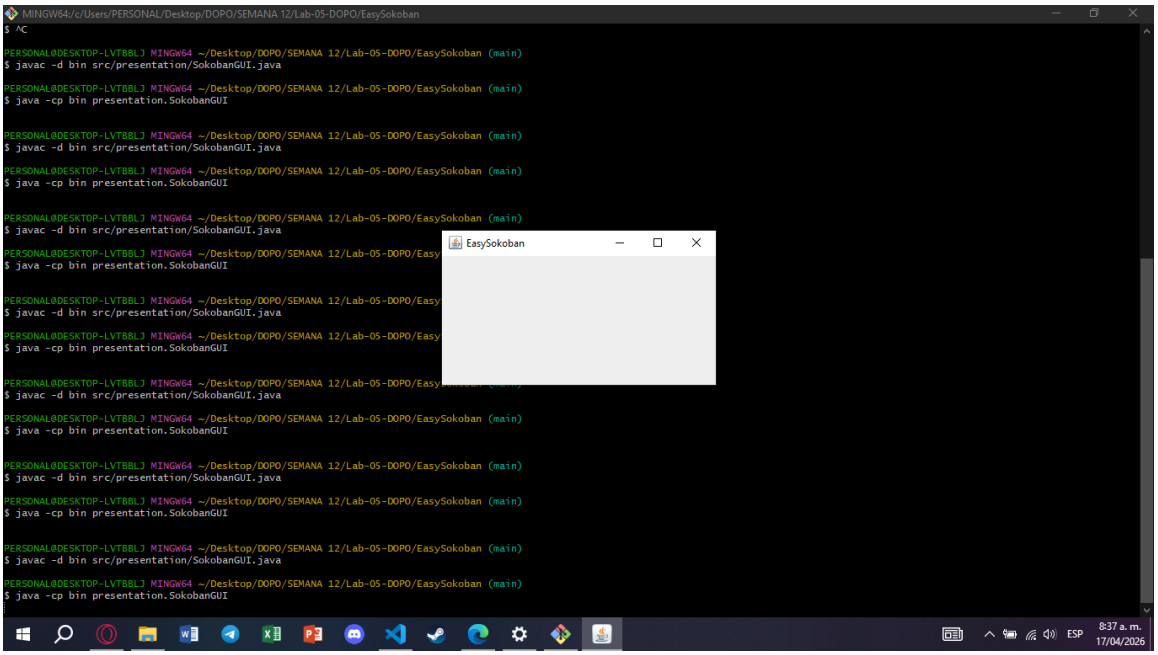
```
public static void main(String[] args){
    SokobanGUI sokoban = new SokobanGUI();
    sokoban.setVisible(b: true);
}
```



2. **Modifiquen el tamaño de la ventana para que ocupe un cuarto de la pantalla y ubíquenla en el centro. Para eso inicien la codificación del método prepareElements. Capturen esa pantalla.**

```java
public void prepareElements(){
    Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
    int width = (int) screenSize.width / 4;
    int height = (int) screenSize.height / 4;
    setSize(width, height);
    setLocationRelativeTo(c: null);

}
```



3. **Traten de cerrar la ventana. ¿Termina la ejecución? ¿Qué deben hacer en consola para terminar la ejecución?**
   Se cierra la ventana, pero no termina la ejecución, es necesario oprimir ctrl + C en consola para que se termine la ejecución.

4. **Estudien en JFrame el método setDefaultCloseOperation. ¿Para qué sirve? ¿Cómo lo usarían si queremos confirmar el cierre de la aplicación? ¿Cómo lo usarían si queremos simplemente cerrar la aplicación?**
   - ¿Para qué sirve?: setDefaultCloseOperation define qué pasa cuando el usuario presiona la x de la ventana
   - ¿Cómo se usaría para confirmar el cierre?: Se usa JFrame.DO_NOTHING_ON_CLOSE para que el ícono x no haga nada por defecto, y en su lugar se programa un oyente que muestra un diálogo de confirmación antes de cerrar.
   - ¿Cómo se usaría si queremos simplemente cerrar?: se usa JFRAME.EXIT_ON_CLOSE que termina directamente la aplicación.

5. **Preparen el "oyente" correspondiente al icono cerrar que le pida al usuario que confirme su selección. Para eso inicien la codificación del método prepareActions y el método asociado a la acción (exit). Ejecuten el programa y cierren el programa. Capturen las pantallas**

```java
public void prepareActions() {
    addWindowListener(new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            exit();
        }
    });
}

public void exit() {
    int respuesta = JOptionPane.showConfirmDialog(this, message: "Do you want to exit the game?", title: "Confirm", JOptionPane.YES_NO_OPTION)
    if (respuesta == JOptionPane.YES_OPTION) {
        System.exit(status: 0);
    }
}
```

C. **Ciclo 1: Ventana con menú - Salir**

El objetivo es implementar un menú clásico para la aplicación con un final adecuado desde la opción del menú para salir. El menú debe ofrecer mínimo las siguientes opciones Nuevo, Abrir– Salvar y Salir. Incluyan los separadores de opciones.

1. **Expliquen los componentes visuales necesarios para este menú. ¿Cuáles serían atributos y cuáles podrían ser varibles del método prepareElements? Justifique.**
   - JMenuBar: Se adjunta al frame y podría necesitarse para habilitarla o modificarla desde otros métodos
   - JMenu: Podría necesitarse habilitarlo/deshabilitarlo según el estado del juego
   - JMenuItem: representa cada opción seleccionable dentro de un JMenu. Se necesitan cuatro: Nuevo, Abrir, Salvar y Salir, con separadores entre grupos lógicos (por ejemplo, entre Salvar y Salir), añadidos mediante addSeparator().

2. **Construya la forma del menú propuesto (prepareElements- prepareElementsMenu). Ejecuten. Capturen la pantalla.**

3. **Preparen el "oyente" correspondiente al icono cerrar con confirmación (prepareActions- prepareActionsMenu). Ejecuten el programa y salgan del programa. Capturen las pantallas.**

REFERENCIAS:

Oracle. (2024). *JFrame (Java SE 11 & JDK 11)*. Oracle Documentation. https://docs.oracle.com/en/java/javase/11/docs/api/java.desktop/javax/swing/JFrame.html

Oracle. (2023). *How to Use Menus*. The Java™ Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/components/menu.html